What is a "dollar" class?

Appendix

An anonymous class is declared and created in one expression, without a name.

- `javac` still names it: `Outer$1`, `Outer$2`, …
- Each one gets its own `Outer$1.class` file.
- Named inner and local classes use `$` names too.

"Dollar" classes in code

Appendix

```
public class Shaker
{
    boolean hasMatch(List<Integer> list, Integer target)
    {
        return list.stream().anyMatch(new Predicate<Integer>()  // Shaker$1
        {
            @Override
            public boolean test(Integer each)
            {
                return each.equals(target);
            }
        });
    }
}
```

- Compare with lambdas (06): a non-capturing lambda is created only once.

What is object pooling?

Appendix

A pool keeps objects and hands them out again instead of allocating new ones.

- Callers return an object to the pool when they are done.
- **Worth it** for costly or limited resources: threads, connections, direct buffers.
- **Rarely worth it** for small, short-lived objects:
  - young-generation allocation is cheap;
  - pooled objects live long and need locking.

What is a compressed OOP?

Appendix

A 4-byte offset that HotSpot stores instead of an 8-byte reference.

- OOP = ordinary object pointer: a reference to a Java object.
- The offset is scaled by the 8-byte object alignment.
- On by default for heaps under 32 GB (`-XX:+UseCompressedOops`).

What is a compact object header?

Appendix

The object header shrinks from 12 bytes to one 8-byte word.

- A header holds the mark word (hash, lock, GC state) and a class pointer.
- Product feature in JDK 25 (JEP 519); experimental in JDK 24 (JEP 450).
- Off by default: `-XX:+UseCompactObjectHeaders`.

What is a stream?

Appendix

A sequence of elements processed by a pipeline: a source, intermediate operations, one terminal operation.

- Intermediate operations (`filter`, `map`) are **lazy**: they only record work.
- The terminal operation (`anyMatch`, `count`, `findFirst`) is **eager**: it runs the pipeline.
- Short-circuiting terminals stop at the first match.
- A stream can be used only once.

A pipeline in code

Appendix

```
boolean found = list.stream()                    // source
                    .filter(each -> each > 500)  // intermediate (lazy)
                    .findFirst()                 // terminal (eager)
                    .isPresent();
```

- Every call builds a new pipeline of objects.
- Nothing runs until `findFirst()`.

Inside: `ReferencePipeline`

Appendix

The internal class behind `Stream<T>`. Primitive versions: `IntPipeline`, `LongPipeline`, `DoublePipeline`.

- `stream()` creates a Head stage and a `Spliterator`.
- Each intermediate operation links one new stage (`StatelessOp` or `StatefulOp`).
- The terminal operation creates a `TerminalOp`, chains `Sink` objects and pushes elements through them.

Eager vs lazy in the JDK

Appendix

	Eager	Lazy
When work happens	At each call	At the terminal operation
In-between results	A new collection per step	None
Short-circuit	Per step	Across the whole chain
JDK examples	`anyMatch`, `count`, `findFirst`, `collect`, `removeIf`, `forEach`	`filter`, `map`, `sorted`

What is a lambda?

Appendix

An anonymous function that implements a functional interface such as `Predicate`.

- No `$` class file: `javac` emits an `invokedynamic` call.
- The runtime (`LambdaMetafactory`) builds the class on first use.
- Whether it allocates depends on what it captures.

Capturing (stateful) lambdas

Appendix

Uses a local, a parameter or `this` from the enclosing scope.

```
boolean hasMatch(List<Integer> list, Integer target)
{
    return list.stream().anyMatch(each -> each.equals(target));  // new object per call
}
```

- Each evaluation creates a new object that holds the captured values.
- Captured locals must be effectively final.

Fat-free (non-capturing) lambdas

Appendix

Uses only its own parameters, constants and static members.

```
boolean hasMatch(List<Integer> list, Integer target)
{
    return Iterate.anySatisfyWith(list, Object::equals, target);  // one shared instance
}
```

- One instance is created and then reused on every call.
- Extra values travel as arguments (the `*With` methods).

What is escape analysis?

Appendix

A C2 JIT check: after inlining, can anyone outside the method see the new object?

- **No**: the allocation is removed; its fields become locals (scalar replacement).
- **Yes**: it is allocated on the heap as usual.
- On by default; `-XX:-DoEscapeAnalysis` turns it off.

Escape analysis in code

Appendix

```
boolean hasMatch(List<Integer> list, Integer target)
{
    return Iterate.anySatisfy(list, each -> each.equals(target));
}
```

- After inlining, the lambda and the loop stay local: 0 B allocated.
- The JDK stream escapes into its own stages: 160 to 224 B per call either way.